ООО «ЕвразТехника ИС»

ПРОЕКТНОЕ РЕШЕНИЕ

Проект «Ruobr Unified Bot»

Информационная система мониторинга школьной успеваемости и питания Ruobr Unified Bot

Документ версии 1.0

Дата: 2025

Конфиденциально

СОДЕРЖАНИЕ

1. Глоссарий, термины и сокращения 2
2. Лист версионности 4
3. Общие положения 5
3.1 Бизнес-контекст и обоснование 5
3.1.1 Основные параметры проекта 6
3.1.2 Цели проекта 6
3.1.3 Задачи проекта 7
3.1.4 RICEF и архетип 8
4. Характеристика объекта автоматизации 9
4.1 Диаграмма бизнес-процесса «To Be» 9
5. Архитектурные решения 11
5.1 Функционально-компонентная архитектура 11
6. Технические решения 13
6.1 Подъём и хранение данных 13
6.2 Базы данных 14
6.3 Backend-компоненты 15
6.4 Frontend-компоненты 16
6.5 Интеграции 17
6.6 Требования к видам обеспечения 17
6.6.1 Программное обеспечение 18
6.6.2 Информационное обеспечение 18
6.6.3 Организационное обеспечение 19
6.6.4 Требования безопасности 19
7. Инфраструктурные решения 20
7.1 Состав технических средств 20
7.2 Используемые сервисы 2

Примечание: для корректного отображения номеров страниц щёлкните правой кнопкой мыши по оглавлению и выберите «Обновить поле».

1. Глоссарий, термины и сокращения

Настоящий раздел содержит определения ключевых терминов, аббревиатур и сокращений, используемых в настоящем проектном решении. Понимание данных терминов необходимо для корректной интерпретации положений документа и однозначного восприятия проектных решений специалистами-разработчиками, аналитиками и заказчиками.

Термин / Аббревиатура	Определение
Ruobr	Информационная система «Региональная оценка качества образования» (cabinet.ruobr.ru), предоставляющая доступ к данным об успеваемости, питании и расписанию школьников Новосибирской области.
TG / Telegram	Мессенджер Telegram — кроссплатформенное облачное приложение для мгновенного обмена сообщениями, предоставляющее API для создания ботов.
VK / ВКонтакте	Социальная сеть ВКонтакте, предоставляющая API для взаимодействия с пользователями и создания сообществ.
Бот (Bot)	Программный агент, взаимодействующий с пользователем через интерфейс мессенджера (Telegram или VK) в диалоговом режиме.
FSM (Finite State Machine)	Конечный автомат — модель вычисления, используемая для управления многошаговыми диалоговыми сценариями в рамках бота.
LRU Cache	Кэш с 淘汰 ом наименее используемых элементов (Least Recently Used) — стратегия управления кэшем, удаляющая элементы, к которым обращались давно.
TTL (Time To Live)	Время жизни — период, в течение которого кэшированные данные считаются актуальными и могут быть использованы без повторного запроса к источнику.
Fernet	Симметричная схема шифрования на основе AES-128-CBC, обеспечивающая конфиденциальное хранение учётных данных пользователей.
ПФДО	Персональный сертификат дополнительного образования — документ, подтверждающий освоение ребёнком программ дополнительного образования.
API (Application Programming Interface)	Интерфейс прикладного программирования — набор протоколов и инструментов для взаимодействия программных компонентов.
Docker	Платформа контейнеризации, обеспечивающая унифицированную среду развёртывания приложений в изолированных контейнерах.

Термин / Аббревиатура	Определение
SQLite	Лёгкая встраиваемая реляционная база данных, работающая без отдельного сервера и хранящая данные в едином файле.
Rate Limiting	Ограничение частоты запросов — механизм защиты от перегрузки системы, ограничивающий количество запросов от одного пользователя в единицу времени.
Middleware	Промежуточное программное обеспечение — компонент, перехватывающий входящие запросы для выполнения дополнительной обработки (аутентификация, ограничение частоты, логирование).
RICEF	Методология описания функциональных требований: Reports, Interfaces, Conversions, Enhancements, Forms.

2. Лист версионности

Лист версионности отражает историю изменения настоящего проектного решения. Каждая запись фиксирует версию документа, дату внесения изменений, ответственного исполнителя и краткое описание внесённых корректировок. Данный раздел обеспечивает прослеживаемость эволюции проектных решений и позволяет отслеживать согласования между участниками проектной команды.

Версия	Дата	Автор	Описание изменений
1.0	2025-01-15	ЕвразТехника ИС	Initial release — полное проектное решение по системе Ruobr Unified Bot. Включены все разделы: бизнес-контекст, архитектурные и технические решения, инфраструктурное проектирование, требования к видам обеспечения.
—	—	—	Резерв для будущих изменений

3. Общие положения

3.1 Бизнес-контекст и обоснование

В современных условиях цифровизации образовательных процессов родители школьников испытывают потребность в оперативном доступе к информации об успеваемости, питании и расписании детей. Новосибирская область использует информационную систему Ruobr

(cabinet.ruobr.ru), которая предоставляет данные через веб-интерфейс. Однако для своевременного получения уведомлений и быстрого доступа к ключевой информации родителям приходится регулярно авторизовываться в системе через браузер, что неудобно при использовании мобильных устройств.

Проект «Ruobr Unified Bot» решает данную проблему путём создания единого мессенджер-бота, работающего одновременно в Telegram и VK. Бот обеспечивает мгновенный доступ к данным системы Ruobr непосредственно из привычной среды мессенджера, а также реализует систему фоновых уведомлений о критических событиях: низком балансе питания, новых оценках, изменениях в меню и днях рождениях одноклассников. Это существенно повышает оперативность получения информации и снижает когнитивную нагрузку на родителей.

Обоснование выбора платформ: Telegram и VK являются наиболее популярными мессенджерами в России, что обеспечивает максимальный охват целевой аудитории. Единый бэкенд для обеих платформ минимизирует затраты на разработку и поддержку, а механизм кросс-линковки аккаунтов позволяет пользователям бесшовно переключаться между платформами без потери настроек и данных.

3.1.1 Основные параметры проекта

Параметр	Значение
Наименование проекта	Ruobr Unified Bot
Полное наименование	Информационная система мониторинга школьной успеваемости и питания Ruobr Unified Bot
Заказчик	ООО «ЕвразТехника ИС»
Тип системы	Мессенджер-бот (Telegram + VK)
Архитектурный паттерн	Backend-only, messaging bot architecture
Основной язык разработки	Python 3.12
СУБД	SQLite (aiosqlite, async)
Контейнеризация	Docker / docker-compose
Часовой пояс	Asia/Novosibirsk (GMT+7)

3.1.2 Цели проекта

Основными целями проекта являются: повышение доступности образовательной информации для родителей путём интеграции с популярными мессенджерами; автоматизация мониторинга ключевых показателей (баланс питания, оценки, расписание) с системой push-уведомлений; обеспечение безопасности хранения учётных данных с использованием криптографического шифрования; создание масштабируемой и поддерживаемой кодовой базы, допускающей расширение функциональности и подключение дополнительных платформ.

3.1.3 Задачи проекта

№	Задача	Описание
1	Аутентификация	Реализация безопасного механизма хранения и расшифровки учётных данных Ruobr с использованием шифрования Fernet (AES-128-CBC)
2	Мониторинг баланса	Отслеживание баланса лицевого счёта питания с настраиваемыми пороговыми значениями и автоматическими уведомлениями при снижении
3	Расписание и ДЗ	Предоставление актуального расписания занятий на сегодня и завтра, а также списка домашних заданий по предметам
4	Мониторинг оценок	Своевременное информирование о новых оценках и изменении средней успеваемости с историческим трекингом
5	Информация о питании	Отображение меню школьной столовой на текущий день с детализацией по приёмам пищи
6	Уведомления о ДР	Фоновый мониторинг дней рождения одноклассников с настраиваемыми заранее уведомлениями
7	Доп. образование (ПФДО)	Отображение программ дополнительного образования, освоенных ребёнком, с данными сертификата ПФДО
8	Кросс-линковка TG ↔ VK	Механизм связывания аккаунтов пользователя в Telegram и VK для единого профиля и синхронизации настроек
9	Защита от перегрузки	Реализация middleware Rate Limiting с ограничением 30 запросов в минуту (sliding window)

3.1.4 RICEF и архетип

Категория RICEF	Компоненты
R (Reports)	Уведомления: баланс, оценки, питание, дни рождения. Исторические данные уведомлений (dedup).
I (Interfaces)	Интеграция с Ruobr REST API, Telegram Bot API (aiogram 3.x), VK API (vkbottle

Категория RICEF	Компоненты
	4.x).
C (Conversions)	Автоматическое шифрование/дешифрование учётных данных при сохранении и чтении из БД.
E (Enhancements)	Фоновая служба уведомлений, LRU-кэш с TTL, кросс-линковка аккаунтов TG ↔ VK.
F (Forms)	Многошаговые диалоги (FSM): авторизация, настройка порогов, привязка аккаунтов, настройка уведомлений.

Архетип пользователя: Родитель школьника (возраст 25–50 лет), активно использующий мессенджеры Telegram и/или VK для повседневной коммуникации. Пользователь ожидает быстрый и интуитивно понятный интерфейс, минимальное количество шагов для получения нужной информации и своевременные уведомления о критических событиях, связанных с обучением и питанием ребёнка.

4. Характеристика объекта автоматизации

4.1 Диаграмма бизнес-процесса «To Be»

Настоящий раздел описывает целевое состояние бизнес-процессов (To Be), которое достигается после внедрения системы Ruobr Unified Bot. Автоматизируемый бизнес-процесс охватывает полный цикл взаимодействия родителя с информационной системой школы: от первоначальной регистрации и настройки до повседневного мониторинга и получения уведомлений.

Процесс начинается с первого обращения пользователя к боту через Telegram или VK. Пользователь инициирует команду авторизации, после чего бот запускает многошаговый диалог (FSM), в ходе которого родитель предоставляет логин и пароль от системы Ruobr. Учётные данные шифруются с использованием алгоритма Fernet (AES-128-CBC) и сохраняются в базе данных SQLite в зашифрованном виде. Дешифрация выполняется только в момент обращения к API Ruobr, что минимизирует риск компрометации.

После успешной авторизации пользователь получает доступ к основному меню бота, включающему следующие функциональные блоки: «Баланс» — отображение текущего баланса лицевого счёта питания с детализацией последних пополнений и списаний; «Расписание» — расписание занятий на текущий и следующий учебный день; «Оценки» — список последних оценок с фильтрацией по предметам и периодам; «Питание» — меню школьной столовой на текущий день; «Дни рождения» — список одноклассников с ближайшими днями рождения; «Доп. образование» — программы дополнительного образования (сертификат ПФДО).

Параллельно с интерактивными запросами работает фоновая служба уведомлений (Background Notification Service). Данная служба выполняет периодический опрос системы Ruobr по расписанию и сравнивает полученные данные с предыдущими значениями, хранящимися в кэше. При обнаружении изменений (новые оценки, снижение баланса ниже порога,

обновление меню) формируется и отправляется уведомление пользователю через выбранный канал (Telegram или VK). Механизм дедупликации (notification_history) предотвращает повторную отправку одинаковых уведомлений.

Отдельным подпроцессом является кросс-линковка аккаунтов. Пользователь может инициировать привязку аккаунта из одной платформы к аккаунту в другой. Система генерирует одноразовый код, который пользователь вводит в целевом мессенджере. После подтверждения оба аккаунта связываются в едином профиле, обеспечивая синхронизацию настроек уведомлений и пороговых значений.

Все запросы пользователей проходят через цепочку middleware: Rate Limiting (ограничение 30 запросов/мин, алгоритм скользящего окна) → Logging (фиксация всех входящих запросов и ответов) → Auth (инъекция user_config в контекст обработки). Данная архитектура обеспечивает отказоустойчивость, безопасность и аудитоспособность системы.

5. Архитектурные решения

5.1 Функционально-компонентная архитектура

Архитектура системы Ruobr Unified Bot построена по принципу модульного монолита с чётким разделением ответственности между компонентами. В качестве базового архитектурного паттерна выбрано Backend-only messaging приложение без выделенного фронтенда — пользовательский интерфейс формируется средствами мессенджера (inline-клавиатуры, форматированные сообщения). Данный подход существенно упрощает развёртывание и обслуживание системы, а также eliminates необходимость создания и поддержки отдельного веб-интерфейса.

Ядром системы является модуль интеграции с API Ruobr (ruobr_client.py), реализованный на основе асинхронного HTTP-клиента httpx. Модуль обеспечивает полную инкапсуляцию взаимодействия с внешней системой: авторизацию через API Ruobr, получение данных об успеваемости, питании, расписании и дополнительных программах. Все внешние вызовы защищены механизмом exponential backoff с configurable числом повторных попыток, что обеспечивает устойчивость к временным сбоям внешнего API.

Слой данных организован на основе SQLite с пулом соединений (5 соединений, aiosqlite). Модуль database.py реализует полный набор CRUD-операций и обеспечивает автоматическую миграцию схемы при запуске приложения. Все чувствительные данные (логин и пароль пользователя) шифруются модулем encryption.py перед сохранением в базу и дешифруются модулем credentials.py при необходимости обращения к API Ruobr.

Модуль кэширования (cache.py) реализует in-memory LRU-кэш с настраиваемым TTL (время жизни). Кэш используется для хранения промежуточных результатов запросов к API Ruobr, что существенно сокращает количество внешних вызовов и повышает скорость обработки пользовательских запросов. Стратегия LRU обеспечивает автоматическую эвикцию наименее используемых записей при достижении предельного размера кэша.

Система обработки уведомлений (notifications.py) функционирует как фоновая служба, выполняющая периодический опрос данных пользователей по заданному расписанию. Служба использует механизм дедупликации через таблицу notification_history для предотвращения повторной отправки идентичных уведомлений. Пользовательские настройки уведомлений

(включение/отключение по категориям, выбор канала доставки) хранятся в таблицах notification_settings и birthday_settings.

Слой взаимодействия с мессенджерами разделён на два параллельных модуля: Telegram-бот (aiogram 3.x) и VK-бот (vkbottle 4.x). Оба модуля используют общие сервисные компоненты (база данных, кэш, API-клиент) и разделяют бизнес-логику, определённую в обработчиках (handlers/). Обработчики реализуют паттерн «thin controller», делегируя бизнес-логику сервисным модулям.

Цепочка middleware обеспечивает сквозную функциональность: RateLimitMiddleware реализует алгоритм скользящего окна с ограничением 30 запросов в минуту на пользователя; AuthMiddleware проверяет наличие аутентификационных данных и инжектирует user_config в контекст обработчика; LoggingMiddleware выполняет централизованное логирование всех входящих запросов и исходящих ответов.

6. Технические решения

6.1 Подъём и хранение данных

Система Ruobr Unified Bot реализует двухуровневую схему подъёма и хранения данных. Первичным источником данных является внешнее REST API системы Ruobr (cabinet.ruobr.ru), через которое бот получает информацию об успеваемости, питании, расписании и дополнительных образовательных программах. Обращение к API осуществляется асинхронно через HTTP-клиент httpx с поддержкой keep-alive соединений и автоматическим повтором запросов при временных сбоях (exponential backoff).

Вторичным уровнем хранения является локальная база данных SQLite, в которой сохраняются: учётные данные пользователей (в зашифрованном виде), настройки уведомлений, пороговые значения для мониторинга баланса, история отправленных уведомлений (для дедупликации), коды кросс-линковки аккаунтов и состояния FSM-диалогов. Данные в SQLite хранятся в единственном файле, который размещается на подключённом Docker-томе для обеспечения персистентности при перезапусках контейнера.

Дополнительный уровень оптимизации — in-memory LRU-кэш с настраиваемым TTL, размещённый в оперативной памяти процесса. Кэш хранит результаты последних запросов к API Ruobr и позволяет обслуживать повторные запросы без обращения к внешней системе. При перезапуске приложения кэш полностью пересоздаётся, что не влияет на целостность данных, поскольку источником истины остаётся SQLite + Ruobr API.

6.2 Базы данных

В качестве системы управления базой данных выбрана SQLite в асинхронной реализации aiosqlite. Выбор обоснован следующими факторами: архитектура приложения не предполагает горизонтального масштабирования (single-server deployment); SQLite обеспечивает достаточную производительность для объёма данных, характерного для мессенджер-бота; встроенная поддержка транзакций и referential integrity; отсутствие необходимости администрирования отдельного сервера БД; компактное хранение данных в единственном файле.

Пул соединений настроен на 5 одновременных соединений, что обеспечивает оптимальный баланс между параллелизмом и потреблением ресурсов. Автоматическая миграция схемы выполняется при каждом запуске приложения: модуль database.py проверяет наличие необходимых таблиц и создаёт отсутствующие. Структура базы данных включает следующие таблицы:

Таблица	Назначение	Ключевые поля
users	Профили пользователей	id, chat_id (TG), peer_id (VK), login, password_encrypted, created_at, updated_at
notification_settings	Настройки уведомлений	user_id, channel (tg/vk), balance/marks/food/birthday_enabled
thresholds	Пороги баланса	user_id, child_id, threshold_balance
fsm_states	FSM состояния (TG)	user_id, state, data (JSON)
vk_fsm_states	FSM состояния (VK)	user_id, state, data (JSON)
notification_history	История уведомлений (dedup)	id, user_id, type, payload_hash, sent_at
birthday_settings	Настройки ДР	user_id, child_id, days_before, enabled
link_codes	Коды привязки TG ↔ VK	code, source_user_id, source_platform, expires_at

6.3 Backend-компоненты

Backend системы полностью реализован на языке Python 3.12 и представляет собой модульное приложение со следующей структурой ключевых компонентов:

Компонент	Файл	Ответственность
Конфигурация	bot/config.py	Загрузка конфигурационных параметров из переменных окружения (.env) через python-dotenv. Валидация обязательных параметров (токены ботов, ключ шифрования, параметры БД).
База данных	bot/database.py	Управление пулом соединений SQLite (5 connections), CRUD-операции для всех таблиц, автоматическая миграция схемы при запуске.
Шифрование	bot/encryption.py	Симметричное шифрование/дешифрование данных с использованием Fernet (AES-128-CBC). Генерация и управление ключами шифрования.
Учётные данные	bot/credentials.py	Безопасное извлечение и дешифрование учётных

Компонент	Файл	Ответственность
		данных пользователя для обращения к API Ruobr. Автоматическая очистка чувствительных данных из памяти.
API-клиент Ruobr	bot/services/ ruobr_client.py	Асинхронный клиент для REST API системы Ruobr. Авторизация, получение данных об успеваемости, питании, расписании. Exponential backoff и повторные попытки.
Уведомления	bot/services/ notifications.py	Фоновая служба периодического мониторинга данных. Формирование и отправка уведомлений. Дедупликация через notification_history.
Кэш	bot/services/cache.py	In-memory LRU-кэш с настраиваемым TTL. Хранение промежуточных результатов запросов для оптимизации производительности.
Обработчики (TG)	bot/handlers/*	Обработчики команд и callback-запросов Telegram-бота: авторизация, баланс, расписание, оценки, питание, дни рождения, ПФДО.
Обработчики (VK)	bot/handlers/*	Общие обработчики для VK-бота, использующие те же сервисные компоненты, что и Telegram-обработчики.
Middleware	bot/middlewares.py	Rate Limiting (30 req/min, sliding window), аутентификация и инъекция user_config, централизованное логирование запросов/ответов.
FSM состояния	bot/states.py	Определение состояний конечных автоматов для многошаговых диалогов: авторизация, настройка порогов, привязка аккаунтов.
Утилиты	bot/utils/*	Форматирование данных, вспомогательные функции для отображения расписания, оценок, баланса и других данных.
Точка входа	main.py	Инициализация и запуск обоих ботов (Telegram + VK) в едином event loop asyncio. Регистрация обработчиков и middleware.

6.4 Frontend-компоненты

Проект Ruobr Unified Bot не предусматривает выделенных Frontend-компонентов в традиционном понимании. Пользовательский интерфейс формируется исключительно средствами мессенджеров Telegram и VK с использованием встроенных возможностей платформ: inline-клавиатуры для навигации и запуска команд, форматированные текстовые сообщения (Markdown/HTML) для отображения структурированных данных, callback-кнопки для

интерактивного взаимодействия, emoji для визуального разделения секций и улучшения восприятия информации.

Данный подход обеспечивает нулевую стоимость разработки и поддержки клиентской части, мгновенную доступность для конечных пользователей без необходимости установки дополнительных приложений, а также естественную адаптивность интерфейса к любому типу устройства (смартфон, планшет, десктоп), поскольку отображение полностью делегировано клиентскому приложению мессенджера.

6.5 Интеграции

Система Ruobr Unified Bot интегрируется с тремя внешними системами через программные интерфейсы (API):

Ruobr REST API (cabinet.ruobr.ru) — основной источник данных об успеваемости, питании, расписании и программах дополнительного образования. Интеграция реализована через асинхронный HTTP-клиент httpx с использованием JSON как формата обмена данными. Авторизация выполняется с использованием учётных данных пользователя (логин/пароль), получаемых из зашифрованного хранилища. Для обеспечения отказоустойчивости реализован механизм exponential backoff с настраиваемым числом повторных попыток. Все запросы выполняются асинхронно, что позволяет обслуживать множество пользователей без блокировки event loop.

Telegram Bot API — канал взаимодействия с пользователями через платформу Telegram. Интеграция реализована через фреймворк aiogram 3.x, который обеспечивает высокоуровневую абстракцию над Telegram Bot API, включая поддержку long polling, встроенную FSM-машину, middleware pipeline и управление inline-клавиатурами. Бот обрабатывает текстовые команды, callback-запросы от inline-кнопок и поддерживает многошаговые диалоги через конечные автоматы.

VK API — канал взаимодействия с пользователями через социальную сеть ВКонтакте. Интеграция реализована через фреймворк vkbottle 4.x, предоставляющий аналогичный aiogram набор абстракций для VK API. Поддерживается обработка текстовых сообщений и callback-кнопок в рамках сообщества ВКонтакте.

6.6 Требования к видам обеспечения

6.6.1 Программное обеспечение

Компонент	Версия	Назначение
Python	3.12	Основной язык разработки
aiogram	3.x	Фреймворк для Telegram-бота
vkbottle	4.x	Фреймворк для VK-бота
aiosqlite	latest	Асинхронный драйвер SQLite
httpx	latest	Асинхронный HTTP-клиент для Ruobr API

Компонент	Версия	Назначение
cryptography	latest	Fernet-шифрование (AES-128-CBC)
ruobr-api	latest	Асинхронный клиент Ruobr API
python-dotenv	latest	Загрузка переменных окружения из .env
Docker Engine	latest	Контейнеризация приложения
docker-compose	latest	Оркестрация контейнеров

6.6.2 Информационное обеспечение

Информационное обеспечение системы включает: структурированную базу данных SQLite с автоматической миграцией схемы; конфигурационные файлы в формате .env для хранения параметров подключения и секретных ключей; журнал событий (логирование) в стандартный вывод Docker (stdout/stderr) для последующей агрегации системами централизованного сбора логов; API-документацию системы Ruobr в формате OpenAPI/Swagger (внешний ресурс).

6.6.3 Организационное обеспечение

Организационное обеспечение включает: документацию по развёртыванию (Dockerfile, docker-compose.yml, deploy.sh); инструкции по настройке переменных окружения и получению токенов ботов; регламент мониторинга работоспособности системы; порядок управления ключами шифрования Fernet (ротация, резервное копирование).

6.6.4 Требования безопасности

Система безопасности Ruobr Unified Bot строится на следующих принципах: шифрование учётных данных пользователей с использованием Fernet (AES-128-CBC) с хранением ключа в переменных окружения; отсутствие хранения паролей в открытом виде (даже в оперативной памяти — дешифрация выполняется только в момент использования с последующей очисткой); rate limiting для защиты от злоупотреблений; дедупликация уведомлений для предотвращения информационного спама; изоляция данных разных пользователей; журнал аудита запросов через LoggingMiddleware.

7. Инфраструктурные решения

7.1 Состав технических средств

Инфраструктура системы Ruobr Unified Bot спроектирована с учётом требований минимальной стоимости владения, простоты эксплуатации и достаточной производительности для целевой аудитории. Система развёртывается в единичном экземпляре (single-instance deployment) на одном физическом или виртуальном сервере, что полностью соответствует масштабу и характеру нагрузки мессенджер-бота.

Компонент	Характеристика	Назначение
Сервер (хост)	1 vCPU, 1–2 ГБ RAM, 10–20 ГБ SSD	Хост-машина для запуска Docker-контейнера с приложением
Docker-контейнер	Образ на базе python:3.12-slim	Изолированная среда выполнения приложения бота
Docker Volume	Монтирование в /app/data	Персистентное хранение файла SQLite и конфигураций
Сетевой интерфейс	host network mode	Прямой доступ к сети хоста для исходящих HTTPS-соединений
Операционная система	Linux (любой дистрибутив с Docker)	Хост-ОС для Docker Engine

Использование Docker-контейнеризации обеспечивает воспроизводимость среды развёртывания, изоляцию от системных библиотек хоста, упрощение обновлений (pull нового образа → перезапуск контейнера), а также возможность миграции между серверами без изменения конфигурации приложения. Режим сети host выбран для минимизации задержек при обращении к внешним API и исключения накладных расходов на NAT.

7.2 Используемые сервисы

В рамках инфраструктурного решения система Ruobr Unified Bot использует следующие внешние и внутренние сервисы:

Сервис	Тип	Описание
Ruobr API (cabinet.ruobr.ru)	Внешний REST API	Источник данных об успеваемости, питании, расписании и программах доп. образования. Протокол HTTPS, формат JSON.
Telegram Bot API	Внешний API	Платформа для доставки сообщений и получения команд от пользователей. Long polling через aiogram.
VK API	Внешний API	Платформа для доставки сообщений в сообществах ВКонтакте. Callback API через vkbottle.
SQLite	Внутренняя СУБД	Локальное хранение пользователей, настроек, FSM-состояний и истории уведомлений. Файл на Docker-томе.
Docker Engine	Инфраструктурн ый	Среда выполнения контейнера приложения.
deploy.sh	CI/CD	Скрипт автоматического развёртывания: остановка текущего контейнера, pull нового образа, запуск с восстановлением томов.

Сервис	Тип	Описание
Cron / asyncio loop	Внутренний	Фоновый планировщик для периодического запуска службы уведомлений.
In-memory LRU Cache	Внутренний	Оптимизация повторных запросов к API Ruobr. TTL-based инвалидация.

Все внешние сервисы доступны через HTTPS, что обеспечивает конфиденциальность передаваемых данных. Система не требует наличия фиксированного IP-адреса или открытия входящих портов (используется long polling для Telegram и callback long polling для VK), что существенно упрощает требования к сетевой инфраструктуре и снижает поверхность атаки.

Скрипт deploy.sh обеспечивает процесс обновления, аналогичный CI/CD pipeline: выполнение миграций базы данных (при необходимости), плавная остановка текущего экземпляра, развёртывание новой версии и запуск с автоматическим подключением persisted Docker-тома. Часовой пояс сервера и контейнера настроен на Asia/Novosibirsk (GMT+7) для корректного отображения расписания и времени отправки уведомлений.

ООО «ЕвразТехника ИС»

Проект «Ruobr Unified Bot»

© 2025. Все права защищены.

Конфиденциально. Без разрешения ООО «ЕвразТехника ИС» воспроизведение или распространение данного документа запрещено.